

M4API: La tecnología de Meta4 como servidor de aplicaciones DCOM



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: enero de 2003.

Fecha de la última publicación: diciembre de 2013.

Tabla de contenidos

Introducción.....	5
Generación de M4API.....	6
Herramienta de generación de M4API: Generador IDL.....	8
Clases de M4API.....	13
Interfaces comunes.....	13
Interfaces específicas.....	15
Interfaz de Meta4Object.....	15
Interfaces de colección.....	16
Interfaz de objeto.....	18
Ejemplos de las interfaces M4API.....	21
Meta4Object SCO_API_HR.....	21
Lista de métodos por nodo:.....	23
Acceso al producto Meta4 desde una aplicación externa.....	25
Aplicación externa: hoja de cálculo Excel.....	25
Meta4Object SCO_API_PERSON_SALARY.....	28
Lista de métodos por nodo:.....	29
Sesiones de M4API.....	31
Producto Meta4 como servidor DCOM.....	33
Soporte de M4API para LATE BINDING.....	35

M4API: Un producto Meta4 como servidor de aplicaciones

Introducción

En la conexión con otras aplicaciones en un entorno cliente/servidor, los productos como Meta4 PeopleNet Ksystem o Meta4 e-mind que se basan en la misma tecnología Meta4, pueden actuar como: cliente o servidor de aplicaciones DCOM. Esto significa que esos productos Meta4 pueden ejecutar aplicaciones que actúan como servidores DCOM, como Excel, Project o Lotus Notes. También se pueden ejecutar desde otras aplicaciones cliente DCOM (Excel, Word, PowerBuilder, Delphi, etc.) o desde cualquier aplicación escrita en Visual Basic.

Las M4API funcionan como puerta de acceso a estos productos Meta4 cuando trabajan como servidor de aplicaciones. En este caso, la tecnología base permite a los productos Meta4 actuar como aplicaciones DCOM mediante bibliotecas OLEDSI. Los objetos que se muestran al exterior son los Meta4Objects que componen el producto Meta4. Las M4API son las interfaces que permiten el acceso a dichos Meta4Objects desde aplicaciones externas, API (*Application Programming Interface*).

A través de los Meta4Objects, las M4API proporcionan acceso tanto a los datos contenidos en los Meta4Objects como a los procesos de los productos Meta4. Las M4API son independientes de los lenguajes de programación y de los mecanismos de comunicación, por lo que constituyen la herramienta perfecta para la integración de componentes, incluidos los desarrollados por clientes y distribuidores de los productos Meta4.

Generación de M4API

Para configurar un Meta4Object de modo que se muestre a clientes DCOM, siga estos pasos:

1. Seleccione el Meta4Object que se va a exponer en aplicaciones externas.
2. Genere el archivo IDL (*Interface Description Language*) correspondiente al Meta4Object seleccionado desde el Diseñador de Meta4Objects. El archivo IDL es un archivo de texto en el que se describe la estructura pública de un Meta4Object. Los archivos IDL generados se guardan por defecto en el directorio especificado como temporal *C:\TEMP* del puesto cliente Su nombre es el código identificativo del Meta4Object y su extensión es *.IDL*.

Figura 1. Archivos IDL, TLB y REG en el directorio C:\TEMP

Name	Size	Type
STD_MT_LU_LOCATION_TYPE.tlb	3KB	Type Library
STD_MT_LU_LOCATION_TYPE.REG	1KB	Registration Entries
STD_MT_LU_LOCATION_TYPE.IDL	3KB	IDL File
STD_MT_LU_LOCATION_TYPE.h	16KB	C Header File
STD_MT_GEO_PLACESession.html	2KB	HTML Document
STD_MT_GEO_PLACEIndex.html	1KB	HTML Document
STD_MT_GEO_PLACE.ic	2KB	C Source File
STD_MT_GEO_PLACE.tlb	3KB	Type Library
STD_MT_GEO_PLACE.REG	1KB	Registration Entries
STD_MT_GEO_PLACE.IDL	3KB	IDL File
STD_MT_GEO_PLACE.h	15KB	C Header File
STD_MT_GEO_DIVSession.html	2KB	HTML Document
STD_MT_GEO_DIVIndex.html	1KB	HTML Document
STD_MT_GEO_DIV.ic	2KB	C Source File
STD_MT_GEO_DIV.tlb	3KB	Type Library
STD_MT_GEO_DIV.REG	1KB	Registration Entries
STD_MT_GEO_DIV.IDL	3KB	IDL File
STD_MT_GEO_DIV.h	15KB	C Header File
STD_MT_COUNTRYSession.html	2KB	HTML Document
STD_MT_COUNTRYIndex.html	1KB	HTML Document
STD_MT_COUNTRY.ic	2KB	C Source File
STD_MT_COUNTRY.tlb	4KB	Type Library
STD_MT_COUNTRY.REG	1KB	Registration Entries
STD_MT_COUNTRY.IDL	3KB	IDL File
STD_MT_COUNTRY.h	15KB	C Header File
SCD_API_PERSON_SALARYSession.html	2KB	HTML Document
SCD_API_PERSON_SALARYIndex.html	2KB	HTML Document
SCD_API_PERSON_SALARY.ic	2KB	C Source File

3. El Generador IDL también puede generar documentación en forma de archivos HTML, en los que se incluye una relación de las propiedades y los métodos que expone el Meta4Object a las aplicaciones cliente.
4. Para crear un Meta4Object al que se pueda tener acceso desde un cliente DCOM, compile el código IDL, de modo que se genere un archivo compilado de tipo TLB (*Type Library* o biblioteca de tipos).
5. Además de crear el archivo TLB, el sistema genera un archivo REG, que permite incluir el Meta4Object en el registro del cliente Windows. Los archivos TLB y REG se guardan también en el directorio *C:\TEMP*, y tienen como nombre el código identificativo del Meta4Object y como extensión *.TLB* y *.REG*, respectivamente.



Cada vez que se modifique un Meta4Object externo, habrá que volver a generar su archivo IDL, compilarlo y registrarlo. Los archivos IDL, TLB, REG y HTML de los Meta4Objects de propiedad pública se cargan en el proceso de instalación estándar de los productos Meta4.

Los productos Meta4 incluyen un conjunto de Meta4Objects de propiedad pública para los que ya se habrán generado y compilado sus respectivos archivos IDL. Este conjunto de Meta4Objects, que componen la interfaz externa de los productos Meta4, es lo que se conoce como M4API.

Además, las versiones más recientes del producto Meta4 incluyen *M4API extendidas*, que permiten generar archivos IDL a partir de Meta4Objects tanto privados como públicos. Con esta funcionalidad, ya no es imprescindible definir con propiedad Pública aquellos Meta4Objects que se desea exponer en aplicaciones externas.

Herramienta de generación de M4API: Generador IDL

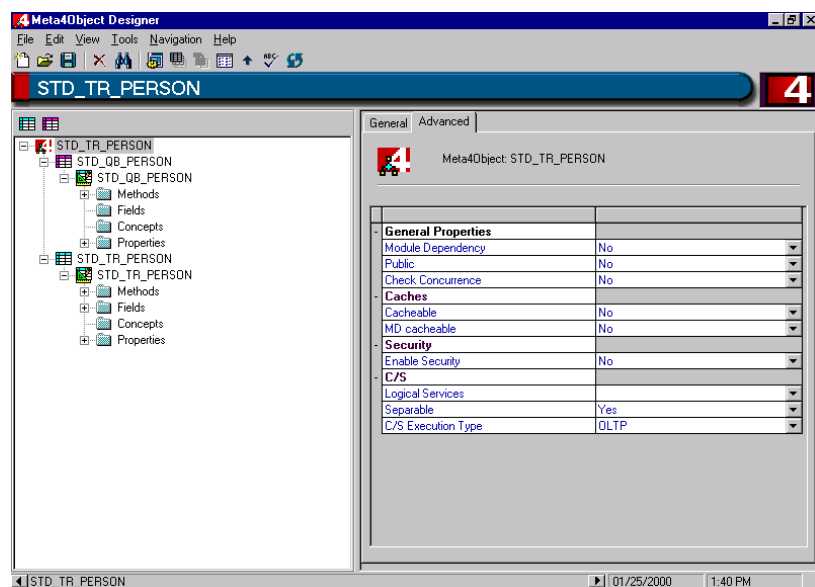
Para mostrar un Meta4Object en aplicaciones externas, la aplicación incluye una herramienta, el Generador IDL, desde la cual se pueden generar archivos IDL, TLB y REG.

Esta herramienta se incluye en el Diseñador de Meta4Objects: para tener acceso, seleccione en el árbol de menú **Herramientas de desarrollo | Meta4Objects | Diseñador de Meta4Objects**, situado a la izquierda de la ventana de inicio del producto Meta4. Haga doble clic en **Diseñador de Meta4Objects**.

Al cargar el Diseñador de Meta4Objects, aparece un formulario en el que puede optar por crear un nuevo Meta4Object o trabajar con uno ya existente. Seleccione uno existente de la lista mostrada al seleccionar la opción **Existente** y hacer clic en el botón **Cargar lista**.

Con independencia de la opción elegida, la pantalla que aparece es la siguiente:

Figura 2. Diseñador de Meta4Objects

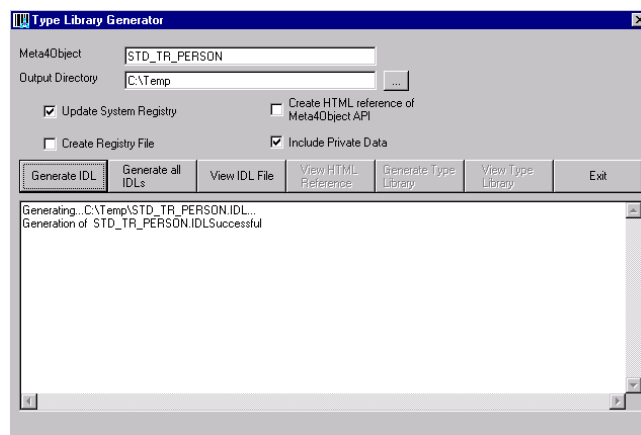


Con la nueva funcionalidad proporcionada por las M4API extendidas, para tener acceso al Generador IDL ya no es requisito imprescindible que el Meta4Object correspondiente esté definido con la propiedad Pública. No obstante, es recomendable conocer el valor de la propiedad para el Meta4Object seleccionado, dado que el proceso de generación del archivo IDL difiere ligeramente de un caso a otro.

En la tabla de propiedades que aparece bajo la pestaña **Avanzado** del Diseñador de Meta4Objects, puede comprobar si un Meta4Object está definido con la propiedad Pública, así como editar y modificar el valor de dicha propiedad.

Después de abrir el Meta4Object desde el Diseñador de Meta4Objects, seleccione en la barra de herramientas **Herramientas | Ir a Generador IDL** para acceder al formulario **Generador IDL**. Utilice este formulario para generar el archivo IDL correspondiente al Meta4Object seleccionado. Previamente, debe haber definido los siguientes parámetros:

Figura 3. Generador IDL



- **Meta4Object:** es el nombre del Meta4Object seleccionado, lo que permite identificar correctamente el archivo IDL generado. El nombre aparece asignado por defecto, si bien es posible modificarlo.
- **Directorio de salida:** la carpeta por defecto es *C:/TEMP*. Se puede modificar, pero es conveniente almacenar todos los archivos generados en el mismo directorio.



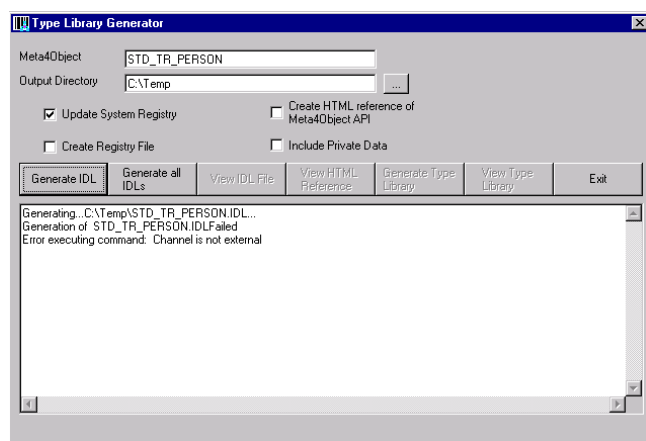
Los parámetros *Meta4Object* y *Directorio de salida* son el nombre y el directorio donde se graba el archivo IDL asociado a él.

- **Actualizar el registro del sistema:** es una opción específica de la interfaz DCOM para que, en caso de que haya archivos IDL, HTML, TLB y REG anteriores para ese mismo Meta4Object, sean sustituidos por los nuevos. Recuerde que, si se produce una modificación en el diseño de un Meta4Object (supresión o modificación de algún elemento, etc.), deben actualizarse los archivos de interfaz asociados a él.

- **Crear HTML de referencia del API M4O:** es la opción que permite generar, en el mismo proceso, archivos HTML e IDL. Los archivos HTML se almacenan en el mismo directorio que los IDL, y contienen listas de propiedades y métodos públicos del Meta4Object correspondiente.
- **Crear archivo de registro:** es la opción DCOM específica para generar archivos de registro, además de los IDL. Los archivos REG permiten incluir el Meta4Object en el registro del cliente Windows, y se almacenan en el mismo directorio que los IDL y los HTML.
- **Incluir datos privados:** es la opción que permite generar archivos IDL a partir de Meta4Objects privados, haciendo uso de la funcionalidad aportada por las M4API extendidas. Esta opción permite obtener archivos IDL, sin necesidad de comprobar que la propiedad Pública del Meta4Object seleccionado se encuentra activada.

No obstante, es conveniente comprobar la propiedad Pública del Meta4Object, dado que la opción **Incluir datos privados** no aparece seleccionada por defecto, y si se intenta generar un archivo IDL a partir de un Meta4Object privado sin seleccionar esta opción, el Generador IDL muestra un mensaje de error: *El Meta4Object seleccionado no es externo.*

Figura 4. Error en la generación del archivo IDL



Una vez definidos los parámetros de generación del archivo IDL y seleccionadas las opciones oportunas, haga clic en el botón **Generar IDL** para que se cree el archivo IDL. En la ventana inferior del registro aparecerá el mensaje *Generación de (archivo Meta4Object_XXX.IDL) ejecutada.*

Puede optar también por generar no sólo el archivo IDL para el Meta4Object cargado en la ventana de diseño, sino también, mediante un proceso de generación masiva de archivos IDL, para todos los Meta4Objects que componen la aplicación Meta4Mind Set, de modo que la interfaz se genere completamente en un único proceso. Para ello, haga clic en el botón **Generar todos los IDL.**

Una vez generado el archivo IDL del Meta4Object seleccionado, si pretende exponerse a interfaces del tipo DCOM, deberá generar también un archivo TLB. Este archivo se crea mediante la compilación del código del archivo IDL obtenido previamente. Para la generación de archivos TLB, haga clic en el botón **Generar Type Library**.



Importante: la opción **Generar Type Library** sólo se activa si en el sistema se encuentra cargado el programa Visual C++. Esto se debe a que la compilación del archivo IDL se realiza desde el módulo MIDL de Visual C++.

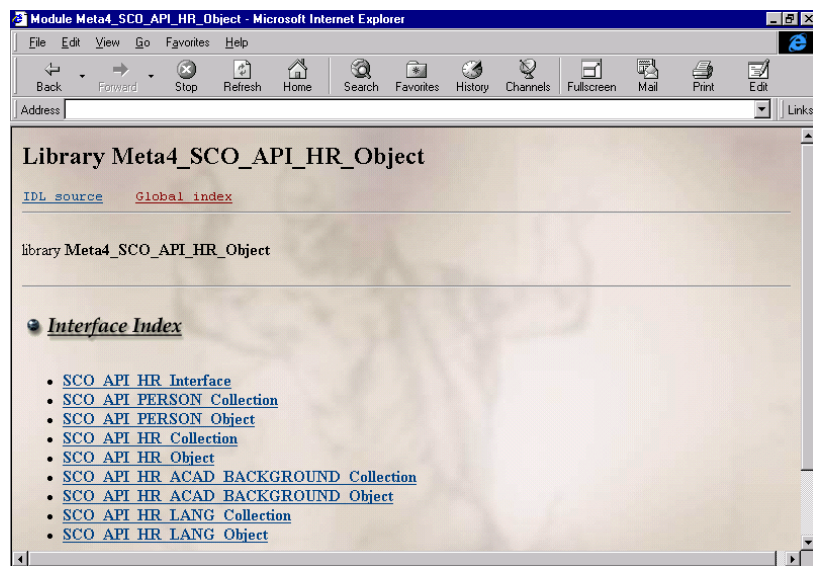
Finalmente, desde el formulario del Generador IDL también se pueden visualizar los archivos IDL y TLB generados. Para ello, haga clic en los botones **Ver archivo de IDL** y **Ver Type Library**, respectivamente.

Figura 5. Vista del archivo IDL desde el Generador IDL

Es importante tener en cuenta que los archivos IDL son dependientes del usuario. Por ello, no es aconsejable compartir o emplear los mismos archivos TLB entre diferentes usuarios, ni siquiera aunque los archivos provengan de los mismos Meta4Objects.

Por otra parte, si en el proceso se han generado también archivos HTML, pueden visualizarse desde un navegador.

Figura 6. Archivo HTML del Meta4Object SCO_API_HR



Clases de M4API

A continuación, se incluye una clasificación de M4API en función de los objetos a los que están asociados. Esta clasificación corresponde a DCOM.

Las M4API se clasifican en dos grandes grupos:

- Interfaces comunes a todos los Meta4Objects
- Interfaces específicas para cada Meta4Object

A su vez, estas últimas se clasifican según tres niveles:

- Meta4Object
- Colección
- Objeto

Interfaces comunes

Se incluyen en este apartado aquellas funciones compartidas por todos los Meta4Objects, esto es, las que tienen que ver con el establecimiento y cierre de sesiones y con la gestión de errores.

- M4API_Exception

Encapsula la información sobre errores. Cada método M4API que devuelva un error, generará una M4API_Exception.

- M4API_SessionManager

Es la puerta de acceso a las M4API: actúa como creador de objetos de sesión.

- M4API_Session

Gestiona la conexión al servidor y encapsula la autenticación del usuario. Incluye las siguientes funciones:

- void Logon(User, Password, Language, LogonServer, PortNumber, UseDefaultRoleFlag, (opcional) SSLCertificate): inicia una nueva sesión. Si la sesión ya está abierta, la reinicia. Si la propiedad UseDefaultRoleFlag está desactivada, deberá ejecutarse previamente la función SetRole. Este parámetro es opcional y sólo se usa si necesita una conexión segura SSL. Si es así, se da el nombre del certificado SSL.
- void Logoff(): cierra una sesión activa. Todos los objetos creados en el curso de la sesión se eliminan automáticamente.

- long GetRoleList(variant): devuelve el número de roles disponible en el Meta4Object Engine. La lista de roles se almacena en una matriz contenida en la variable de tipo *Variant*.
- bool SetRole(RoleName, SetAsDefaultRoleFlag): establece en el Meta4Object Engine el rol incluido como variable. Si la opción de establecimiento por defecto (SetAsDefaultRole Flag) está activada, el rol establecido se convierte en rol por defecto.

A continuación, se incluye un ejemplo de interfaz común de un archivo IDL de tipo DCOM.

EJEMPLO

```
// Meta4 DCOM M4API.IDL file
#include "ChDefine.hpp"

[
    uuid(B09F4950-D9B9-11d1-8C26-00C04FA4988F),
    version(1.0),
    helpstring("Meta4 M4API Object")
]
library Meta4_M4API_Object
{
    importlib("stdole32.tlb");
    [uuid(076BE561-F7A2-11d1-8C43-00C04FA4988F)]
    interface IM4API_Session : IDispatch {
        HRESULT Logon([in] BSTR bstrUsername, [in] BSTR
            ai_bstrPassword, [in] long ai_lLanguage, [in] BSTR
            ai_bstrLogonServer, [in] long ai_lPortNumber, [in]
            VARIANT_BOOL ai_varbUseDefaultRole, [in, optional] BSTR
            ai_stSSL);
        HRESULT Logoff();
        HRESULT GetRoleList([out, retval] VARIANT* ao_pVariant);
        HRESULT SetRole([in] BSTR ai_bstrRoleName, [in] VARIANT_BOOL
            ai_bSetAsDefaultRole);
    };
    [uuid(076BE563-D9B9-11d1-8C26-00C04FA4988F)]
    coclass M4API_Session {
        [default] interface IM4API_Session;
    };
};
```

Interfaces específicas

En este apartado, se incluyen las funciones particulares de cada Meta4Object exportado que están presentes en todas las interfaces de Meta4Object. Por otra parte, los Meta4Objects pueden presentar también funciones específicas propias de los procesos que contienen cada uno de ellos.

Interfaz de Meta4Object

Proporciona la funcionalidad básica para guardar los cambios y tener acceso a la colección raíz de registros.

Para exponer un Meta4Object a aplicaciones externas a través de las M4API, se recomienda desactivar la opción **Carga automática** para dicho Meta4Object.

La interfaz de Meta4Object incluye los siguientes métodos:

- `void Create(void* aSession)`: crea un objeto de tipo Meta4Object. El argumento requerido es la referencia a un de sesión que el usuario debe haber activado correctamente.



El método *Create* es válido únicamente para interfaces DCOM.

- `void Load( )`: carga el Meta4Object especificado sin hacer público su contenido y ejecuta los filtros que se hayan definido sobre la colección raíz.
- `void CommitChanges( )`: guarda los cambios producidos desde la última vez que se ejecutó este método. Trabaja en modo transaccional, es decir: o se realiza completamente, o falla. En caso de fallo, se genera un error. Tenga en cuenta que los cambios realizados sobre los objetos no persistirán a menos que se ejecute esta opción.

A continuación, se incluye un ejemplo de interfaz específica a nivel de Meta4Object de un archivo IDL de tipo DCOM.

EJEMPLO

```
[uuid(FC71A5E4-1B28-11D2-BCBC-00C04FA9BA97)]
dispinterface PERSONAL DATA_Interface {
properties:
    [id(DISPID_NAME), readonly]
    BSTR Name;
    [id(DISPID_ROOT_NODE), readonly]
    SCO_PERSON_Collection* co_SCO_PERSON;

methods:
    [id(DISPID_CREATE)]
    void Create([in] IUnknown* aSession);
    [id(DISPID_LOAD)]
    void Load();
    [id(DISPID_COMMIT)]
    void CommitChanges();
};
```

Interfaces de colección

Incluyen los métodos estándar aplicables a una colección de registros, así como los elementos de nodo y de bloque. Se genera una interfaz de colección para cada nodo que se declare como externo. La interfaz de colección consta de las siguientes propiedades y métodos:

- `unsigned long Count( )`: devuelve el número de elementos que contiene una colección.
- `<Object_Interface> Item(unsigned long index)`: recupera el objeto de la colección que ocupa el lugar que indica el índice pasado como argumento. El rango del índice abarca desde 1 hasta el número de elementos obtenido mediante la función anterior menos uno (`Count`). Se genera un error si el índice especificado está fuera de rango.
- `Add()`: crea un nuevo objeto sin iniciar y lo agrega al final de la colección. Devuelve el nuevo creado.
- `Remove(unsigned long Index)`: elimina el objeto de la colección que se encuentra en la posición indicada por el índice. Esto implica la destrucción de cualquier subcolección contenida en el `Meta4Object`, así como de los

objetos que ésta pueda a su vez contener. Finalmente, la eliminación de los registros queda a cargo del Meta4Object. Se genera un error si el índice especificado está fuera de rango.

- <Collection_Interface> Find(in string Search): devuelve el subconjunto de objetos dentro de la colección que cumplen los criterios LN4 de búsqueda especificados. Tenga en cuenta que este subconjunto no constituye una nueva colección independiente dentro de la anterior, es decir, no contiene instancias duplicadas a los objetos de la colección *padre*, sino tan sólo referencias a los mismos. Al igual que en los métodos anteriores, se genera un error si la búsqueda especificada no es correcta.
- Boolean ExistenceCheck(in string Search): devuelve el valor *Verdadero* si al menos un objeto perteneciente a la colección verifica los criterios LN4 especificados. Se genera un error si la búsqueda especificada no es correcta.

A continuación, se incluye un ejemplo de interfaces específicas a nivel de colección de un archivo IDL de tipo DCOM.

EJEMPLO

```
[uuid(FC71A5EF-1B28-11D2-BCBC-00C04FA9BA97)]
dispinterface SCO_PERSON_Collection {
// properties
properties:
    [id(DISPID_COUNT), readonly]
    long Count;
    [id(0x0f000000)]
    BSTR SYS_SENTENCE_A;
    [id(0x0f000001)]
    BSTR SYS_PARAM_A;
// methods
methods:
    [id(DISPID_EVALUATE)]
    SCO_PERSON_Object* Item([in] long anIndex);
    [restricted, id(DISPID_NEWENUM)]
    IUnknown* _NewEnum();
    [id(DISPID_ADD)]
    SCO_PERSON_Object* Add();
    [id(DISPID_REMOVE)]
    void Remove([in] long anIndex);
    [id(DISPID_FIND)]
    SCO_PERSON_Collection* Find([in] BSTR aSearch);
    [id(DISPID_EXISTCHECK)]
```

```

        HRESULT ExistenceCheck([in] BSTR aSearch, [out, retval]
        VARIANT_BOOL * pBool);
        [id(0x0f000002)]
        long ZOOM([in, out] BSTR *ARG_SEC_TI, [in, out] BSTR
        *ARG_STD_ID_PERSON);
        [id(0x0f000003)]
        [vararg] long LOOK_UP([in, out] BSTR *ARG_APPLYFILTERITEM,
        [in, out] Date
        *ARG_CORENDDATE, [in, out] Date *ARG_CORSTARTDATE, [in, out]
        Date *ARG_ENDDATE, [in, out] BSTR *ARG_SECFILTERTI, [in, out]
        long *ARG_SECONDARYNUMBER, [in, out] Date *ARG_STARTDATE,
        [in] SAFEARRAY(VARIANT) *args);
        [id(0x0f000004)]
        long DESTROYBLOCK();
        [id(0x0f000005)]
        long APPLY_ZOOM([in, out] BSTR *ARG_STD_ID_PERSON, [in, out]
        long *ARG_IS_CACHE);
    };

```

Interfaz de objeto

Por cada interfaz de colección hay una interfaz de objeto. Ésta trabaja con elementos a nivel de registro, tanto campos como métodos.

Las relaciones de nodo RB (registro-bloque) se asignan como objetos compuestos; es decir, para cada nodo hijo se define un atributo en la interfaz de objeto que representa una colección de objetos hijos. Para que un elemento se exporte al archivo IDL, debe declararse con propiedad pública.

La interfaz a nivel de presenta el siguiente método:

- **Remove()**: elimina el objeto de la colección a la que pertenece. Este método equivale a invocar el método de la interfaz de colección.

A continuación, se incluye un ejemplo de interfaces específicas a nivel de objeto de un archivo IDL de tipo DCOM.

EJEMPLO

```
[uuid(FC71A5F0-1B28-11D2-BCBC-00C04FA9BA97)]
dispinterface SCO_PERSON_Object {
// properties
properties:
    [id(0x0f000000)]
    BSTR SUS_SUFFIX;
    [id(0x0f000001)]
    Date SUS_DEATH_DATE;
    [id(0x0f000002)]
    BSTR STD_WELLNESS_IND;
    [id(0x0f000003)]
    BSTR STD_SSN;
    [id(0x0f000004)]
    BSTR STD_PREV_LAST_NAME;
    [id(0x0f000005)]
    BSTR STD_N_USUAL_NAME;
    [id(0x0f000006)]
    BSTR STD_N_MIDDLE_NAME;
    [id(0x0f000007)]
    BSTR STD_N_MAIDEN_NAME;
    [id(0x0f000008)]
    BSTR STD_N_FIRST_NAME;
    [id(0x0f000009)]
    BSTR STD_N_FAMILY_NAME_2;
    [id(0x0f00000a)]
    BSTR STD_N_FAMILY_NAME_1;
    [id(0x0f00000b)]
    BSTR STD_ID_SUB_GEO_DIV;
    [id(0x0f00000c)]
    BSTR STD_ID_SALUTATION;
    [id(0x0f00000d)]
    BSTR STD_ID_PERSON;
    [id(0x0f00000e)]
    BSTR STD_ID_GEO_DIV;
    [id(0x0f00000f)]
    BSTR STD_ID_GENDER;
    [id(0x0f000010)]
```

```
BSTR STD_ID_COUNTRY;
[id(0x0f000011)]
Date STD_DT_BIRTH;
[id(0x0f000012)]
BSTR STD_COMMENT;
[id(0x0f000013)]
BSTR SCO_PHOTO;
[id(0x0f000014)]
BSTR SCO_HOME_PAGE;
[id(0x0f000015)]
long MAX_ID_PERSON;
[id(0x0f000016)]
BSTR EXECUTESQL;
[id(0x0f000017)]
Date DT_LAST_UPDATE;
[id(0x00000001)]
SCO_ADDRESS_Collection* co_SCO_ADDRESS;
[id(0x00000002)]
SCO_EMAIL_Collection* co_SCO_EMAIL;
[id(0x00000004)]
SCO_PHONE_FAX_Collection* co_SCO_PHONE_FAX;
// methods
methods:
[id(DISPID_REMOVE)]
void Remove();
[id(0x0f000018)]
long CALCULATE_ID_PERSON();
[id(0x0f000019)]
long ASSING_ID_PERSON();
};
```

Ejemplos de las interfaces M4API

A continuación, se muestran dos ejemplos básicos de implantación de las M4API, que permiten comprender mejor el modo de aplicación de las funciones explicadas en el documento a un caso práctico.

El primer ejemplo muestra la funcionalidad de las M4API relacionadas con las entidades Persona y Recurso humano. En el segundo, se simula la conexión al producto Meta4 desde una aplicación externa (en este caso, la hoja de cálculo Excel).

Meta4Object SCO_API_HR

Como primer ejemplo, hay un Meta4Object, diseñado específicamente para esta funcionalidad. Este objeto, denominado Recurso humano o SCO_API_HR, contiene los datos relacionados con las personas y los empleados. Puede cargarse desde la herramienta Diseñador de Meta4Objects, a la que se tiene acceso desde el árbol de menús **Herramientas de desarrollo | Meta4Objects | Diseñador de Meta4Objects** y haciendo doble clic en **Diseñador de Meta4Objects**.

El Meta4Object Recurso humano consta de diez nodos. Dispone de cuatro formularios básicos:

- **Persona:** contiene la información directamente relacionada con la persona (dirección, correo electrónico y teléfono). Muestra los valores actuales de esos campos.
- **Recurso humano:** incluye los datos relacionados con el recurso humano (estado civil, nacionalidad y número de hijos). Muestra el estado civil actual y la nacionalidad principal.
- **Idiomas:** incluye los datos sobre los idiomas que habla la persona.
- **Historial académico:** incluye los datos del historial académico de la persona.

Este Meta4Object posee, además, seis filtros predefinidos:

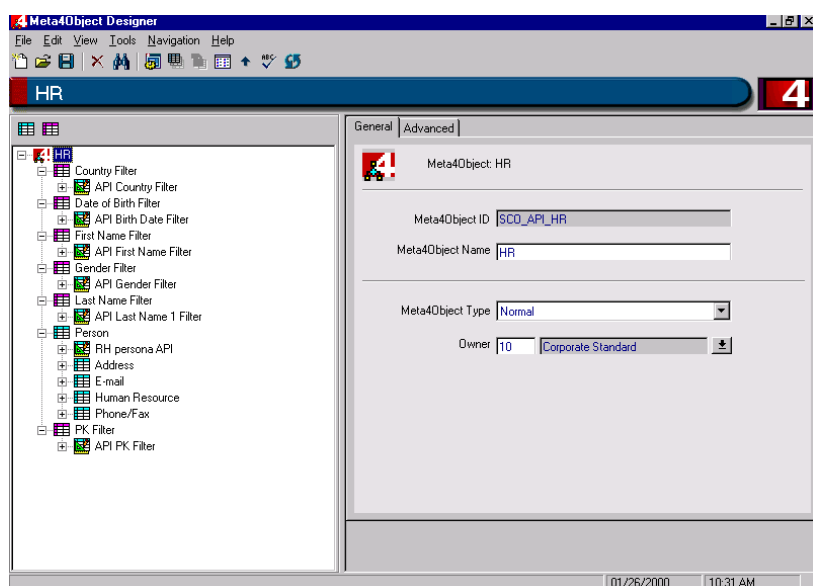
- **SCO_API_PK_FILTER:** filtro de registros por la clave primaria (ID_PERSON), de modo que carga un único registro
- **SCO_API_COUNTRY_FILTER:** filtro que carga las personas nacidas en el país que se incluye como argumento. Se trata de un filtro QBF (*Query Basic Filter*, definido por defecto).

- **SCO_API_DATE_BIRTH_FILTER**: filtro de los registros por la fecha de nacimiento; sólo se cargan aquellas personas cuya fecha de nacimiento sea posterior a la indicada. Se trata de un filtro QBF.
- **SCO_API_GENDER_FILTER**: filtra registros por sexo (M/F). Se trata de un filtro QBF.
- **SCO_API_NAME_FILTER**: filtro de las personas por nombre de pila. Se trata de un filtro QBF.
- **SCO_API_FAMILY_NAME_1_FILTER**: filtro de las personas por apellido. Se trata de un filtro QBF.

A continuación, se muestra la estructura interna del Meta4Object, con los nodos que lo componen y los métodos públicos asociados a cada uno de ellos.

- Nodo: **SCO_API_PERSON**, nodo externo
- Nodo: **SCO_API_ADDRESS**, nodo interno
- Nodo: **SCO_API_EMAIL**, nodo interno
- Nodo: **SCO_API_PHONE_FAX**, nodo interno
- Nodo: **SCO_API_HR**, nodo externo
- Nodo: **SCO_API_HR_NATIONALITY**
- Nodo: **SCO_API_HR_MAR_STAT**
- Nodo: **SCO_API_HR_CHILD_EL**
- Nodo: **SCO_API_HR_LANG**
- Nodo: **SCO_API_HR_ACAD_BACKGROUND**

Figura 7. Meta4Object público SCO_API_HR



Lista de métodos por nodo:



Los nodos resaltados en negrita son los que se han definido como externos.

- **SCO_API_PERSON** (nodo raíz)
 - SCO_CREATE_ADDRESS: método de propiedad pública con alcance de registro que vacía las propiedades mostradas en pantalla, de modo que es posible introducir una nueva dirección. Tenga en cuenta que es necesario guardar tras introducir los nuevos datos para validar los cambios en la base de datos, lo que sucederá al ejecutarse el método SCO_PERSIST_PERSON.
 - SCO_CHANGE_ADDRESS: método de propiedad pública con alcance de registro, que prepara el Meta4Object para modificar los datos que aparecen en pantalla. Todos los datos modificados se sobrescribirán en la base de datos cuando se ejecute el método SCO_PERSIST_PERSON.
 - SCO_CREATE_MAIL: método de propiedad pública con alcance de registro, similar en su funcionalidad a SCO_CREATE ADDRESS.
 - SCO_CHANGE_MAIL: método de propiedad pública con alcance de registro, similar en su funcionalidad a SCO_CHANGE ADDRESS.
 - SCO_CREATE_PHONE: método de propiedad pública con alcance de registro, similar en su funcionalidad a SCO_CREATE ADDRESS.
 - SCO_CHANGE_PHONE: método de propiedad pública con alcance de registro, similar en su funcionalidad a SCO_CHANGE ADDRESS.
 - SCO_PERSIST_PERSON: método de propiedad pública con alcance de registro, que guarda los datos en la base de datos.
- **Nodo SCO_API_ADDRESS**
 - SCO_CURRENT_ADDRESS: método privado con alcance de bloque, que recupera la información de la dirección actual de la persona y la carga en los campos de propiedad pública de la estructura de nodo SCO_API_PERSON_HR.
- **Nodo SCO_API_EMAIL**
 - SCO_CURRENT_EMAIL: método privado con alcance de bloque, que recupera los datos del correo electrónico actual de la persona y los carga en los campos de propiedad pública de la estructura de nodo SCO_API_PERSON_HR.
- **Nodo SCO_API_PHONE_FAX**

- **SCO_CURRENT_PHONE**: método privado con alcance de bloque, que recupera los datos del teléfono actual de la persona y los carga en los campos de propiedad pública de la estructura de nodo **SCO_API_PERSON_HR**.

■ **Nodo **SCO_API_HR****

- **SCO_CHANGE_MAR_STAT**: método de propiedad pública con alcance de registro que prepara el **Meta4Object** para modificar los datos de estado civil que aparecen en pantalla. Los datos modificados se sobrescribirán en la base de datos cuando se ejecute el método **SCO_PERSIST_HR**.
- **SCO_PERSIST_HR**: método de propiedad pública con alcance de registro que guarda las modificaciones anteriores en la base de datos.
- **SCO_CHANGE_NATIONALITY**: método de propiedad pública con alcance de registro que prepara el **Meta4Object** para modificar los datos que aparecen en pantalla. Este método cambia la nacionalidad de la persona y pasa a considerarla la nacionalidad principal en caso de que ya exista. Los datos modificados se sobrescribirán en la base de datos cuando se ejecute el método **SCO_PERSIST_HR**.

■ **Nodo **SCO_API_HR_NATIONALITY****

- **SCO_NATIONALITY**: método privado con alcance de bloque, que recupera los datos de nacionalidad principal de la persona y los carga en los campos de propiedad pública de la estructura de nodo **SCO_API_PERSON_HR**.

■ **Nodo **SCO_API_HR_MAR_STAT****

- **SCO_CURRENT_MAR_STAT**: método privado con alcance de bloque, que recupera los datos de estado civil de la persona y los carga en los campos de propiedad pública de la estructura de **SCO_API_PERSON_HR**.

■ **Nodo **SCO_API_CHILD_EL****

- **SCO_CALC_CHILDREN**: método privado con alcance de bloque, que cuenta el número de hijos que la persona tiene y carga el dato en el campo de propiedad pública correspondiente de la estructura de nodo **SCO_API_HR**.

■ **Nodo **SCO_API_HR_LANG****

- **Nodo **SCO_API_HR_ACAD_BACKGROUND****: este nodo, al igual que el anterior, es externo. Ambos corresponden respectivamente a las estructuras de nodo de mantenimiento de idiomas e historiales académicos, y no presentan métodos.

Acceso al producto Meta4 desde una aplicación externa

En este segundo ejemplo de la funcionalidad de las M4API, se tiene acceso al producto Meta4 desde una aplicación externa que invoca a un Meta4Object definido con propiedad pública. Desde la aplicación externa, puede leer los datos almacenados en el Meta4Object y ejecutar los métodos éste contiene. Por tanto, hay dos elementos principales:

- La aplicación externa, que es una hoja de cálculo Excel
- El Meta4Object, invocado desde la hoja Excel, que es el SCO_API_PERSON_SALARY

Aplicación externa: hoja de cálculo Excel

La hoja de cálculo Person.xls recupera algunos datos de la persona (código identificativo, nombre, apellidos, nombre familiar y número de la seguridad social) y algunos de la dirección (código identificativo, líneas 1 a 4 del campo dirección, código postal, ciudad, estado y país). Person.xls consta de dos hojas: Edit_Person y Geo_Analysis.

Desde la hoja Edit_Person, puede realizar las siguientes tareas con los datos recuperados del Meta4Object:

Figura 8. Hoja de cálculo Person.xls

The screenshot shows a Microsoft Excel spreadsheet titled 'Person.xls [Read-Only]'. The active worksheet is 'Edit Person'. The spreadsheet contains a form for entering personal data. The form is titled 'Meta4! PERSONAL DATA' and includes the following fields:

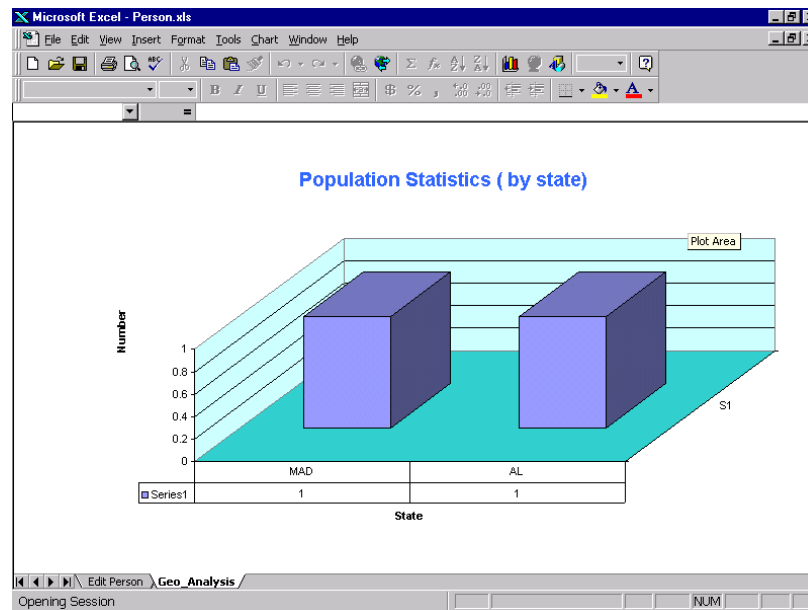
- Person ID: 10060
- First Name: José Pedro
- Family Name: Jiménez
- Usual Name: Pedro
- Age: (empty)
- Address ID: 1
- Address Line 1: C/ Pintor Ribera
- Address Line 2: Nº 2, 4 F
- Address Line 3: (empty)
- Address Line 4: (empty)
- Zip Code: 28016
- City: 28011
- Geographical Place: (empty)
- State: 28
- Country: 34
- Location Type: (empty)

The spreadsheet also shows a status bar at the bottom indicating 'Record: 1 from 39' and a 'NUM' button.

- Navegar por los registros de personas
- Navegar por los registros de direcciones de una persona
- Comprobar la existencia de una persona determinada
- Calcular la edad de una persona determinada
- Calcular la edad media de un grupo de personas filtradas por país y sexo
- Agregar una nueva dirección a una persona determinada
- Modificar los parámetros de una dirección concreta
- Guardar las modificaciones realizadas
- Generar un documento Word desde el producto Meta4 con la dirección de cada persona

Además, con el método externo SCO_ADDRESS_STATE, puede obtener un gráfico en la hoja Geo_Analysis, en el que se represente el número de personas por estados. El gráfico se genera automáticamente al abrir la hoja Geo_Analysis. Para crearlo, se invocan dos Meta4Objects: el básico SCO_API_PERSON_SALARY y el de mantenimiento de los países, STD_MT_GEO_DIV.

Figura 9. Gráfico en la hoja Geo_Analysis de Person.xls



Las M4API se invocan desde la hoja de cálculo Person.xls mediante el lenguaje Visual Basic for Applications (VBA) según el servidor de OLEDSI.

Figura 10. Código Visual Basic de la hoja Person.xls

```

Option Explicit

Public gsession As M4API.Session
Public gchannel As SCO.API.PERSON.SALARY.Channel
Public co_personas As SCO.API.PERSON.Collection
Public gpersona As SCO.API.PERSON.Object
Public co_direcciones As SCO.API.ADDRESS.Collection
Public gdireccion As SCO.API.ADDRESS.Object
Public max_personas As Long
Public persona_actual As Long
Public max_direcciones As Long
Public direccion_actual As Long
Public first_time As Boolean
Public filtrar As Boolean
Public filtroID As String

Public Sub Borrar()

Dim fila As Long
fila = 1
While Sheet3.Cells(fila, 1).Value <> ""
    Sheet3.Cells(fila, 1).Value = ""
    Sheet3.Cells(fila, 2).Value = ""
    fila = fila + 1
Wend

End Sub
    
```

Meta4Object SCO_API_PERSON_SALARY

Se trata de un Meta4Object multinodo con seis nodos, todos ellos definidos como visibles.

- SCO_API_PK_FILTER: filtro de registros por la clave primaria (ID_PERSON), de modo que carga un único registro

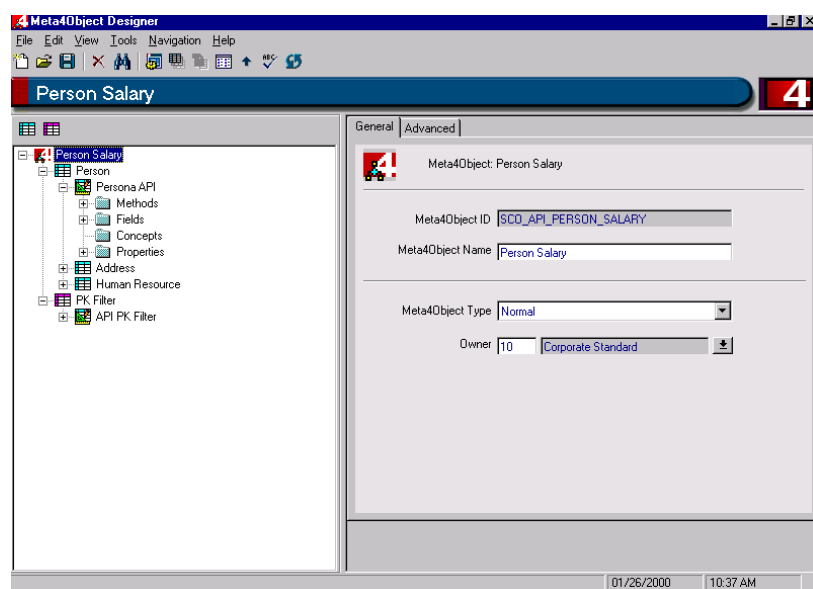
A continuación, se muestra la estructura interna del Meta4Object SCO_API_PERSON_SALARY, con los diferentes nodos que lo componen y los métodos de propiedad pública asociados a cada uno de los nodos.



Los nodos resaltados en negrita son los que se han definido como externos.

- Nodo: **SCO_API_PERSON**, nodo externo
- Nodo: **SCO_API_ADDRESS**, nodo externo
- Nodo: SCO_API_HR
- Nodo: SCO_API_HR_PERIOD
- Nodo: SCO_API_HR_ROLE
- Nodo: SCO_API_ROLE_VALUES

Figura 11. Meta4Object SCO_API_PERSON_SALARY



Lista de métodos por nodo:

- **SCO_API_PERSON** (nodo raíz)
 - SCO_SALARY_INCREASE (número de años, porcentaje): método de propiedad pública con alcance de nodo, con dos argumentos de entrada: número de años y porcentaje. El método aumenta el salario, según el porcentaje indicado, a las personas que llevan trabajando en el mismo rol un número de años igual o superior al establecido. El campo SCO_DT_END de la entidad Salario se modifica y sustituye por la fecha actual, y el nuevo registro de salario se agrega a la tabla SUS_ROLE_VALUES. Los datos modificados no se guardan automáticamente, sino que es preciso invocar el método .
 - SCO_CALC_AVERAGE_AGE (país, sexo): método de propiedad pública con alcance de bloque, con dos argumentos de entrada: país de origen y sexo. Con estas variables, calcula la edad media de las personas que cumplen ambas condiciones, haciendo uso del método privado SCO_CALC_AGE.
 - SCO_PERSON_AGE: método de propiedad pública con alcance de registro, que invoca el método SCO_CALC_AGE e inserta el resultado de la llamada en la propiedad SCO_AGE.
 - SCO_CALC_AGE: método de propiedad pública con alcance de registro, que calcula la edad de la persona seleccionada.
 - SCO_ADDRESS_WORD: método de propiedad pública con alcance de registro, que invoca el procesador Word y genera una carta para cada empleado con su nombre y dirección.
- **Nodo: SCO_API_ADDRESS**, nodo externo
 - SCO_ADDRESS_STATE: método de propiedad pública con alcance de nodo, con un argumento de entrada. El método calcula el número de direcciones por estado entre las direcciones de los domicilios de las personas que el Meta4Object carga como registros. El resultado se muestra en una caja de mensajes, que aparecerá igualmente en la hoja de cálculo Person.xls cuando el método se ejecute desde ésta.
- **Nodo SCO_API_HR**
- **Nodo SCO_API_HR_PERIOD**
- **Nodo SCO_API_HR_ROLE**: al igual que los dos anteriores, este nodo es interno. Los tres nodos corresponden, respectivamente, a las estructuras de nodo de mantenimiento HR, HR Period y HR Role. No presentan métodos.

■ Nodo SCO_API_ROLE_VALUES

- SCO_INCR_PER_ROLE (año): método de propiedad pública con alcance de bloque, con un argumento de entrada: *year*. El método calcula el número de aumentos que una persona ha disfrutado en un rol específico desde el año especificado como argumento. El resultado se muestra en una caja de mensajes, que aparecerá igualmente en la hoja de cálculo Person.xls cuando el método se ejecute desde ésta.

Sesiones de M4API

A continuación, se muestran algunos ejemplos de sesiones M4API sobre el Meta4Object Recurso humano y desde la hoja Person.xls, respectivamente.

Así, una sesión DCOM sobre un Meta4Object como Recurso humano, empleando interfaces estándar, tiene la siguiente estructura, escrita en código Visual Basic:

EJEMPLO

```
' Creación de sesión y conexión al servidor
Dim oSession As M4API_Session
Set oSession = New M4API_Session
oSession.Logon strUser, strPassword, lLanguage, strLogonServer,
lPortNumber, varbUseDefaultRole

' Creación y carga del Meta4Object de trabajo
Dim oCapChannel As PERSONAL_DATA_Channel
Set oCapChannel = New PERSONAL_DATA_Channel
oCapChannel.Load

' Recuperación de la lista de personas (colección Persona)
Dim oPersonCollection As PERSON_Collection
oPersonCollection = oPersonChannel.co_PERSON

' Recuperación de los datos de una persona concreta (registro
seleccionado de la colección Persona)
oPerson = oPersonCollection.Item(0)
Print oPerson.STD_ID_GENDER;
Print oPerson.STD_ID_PERSON;
Print oPerson.STD_N_FAMILY_NAME_1;

' Modificación de los datos del registro seleccionado
oPerson.STD_N_FIRST_NAME = "Michael"
oPerson.STD_N_FAMILY_NAME_1 = "Fanshawe"
If oPersonChannel.CommitChanges() Is OK
Print "Changes succesfully saved"

' Creación de una nueva persona (nuevo registro en la colección)
oNewPerson = oPersonCollection.Add()
```

```
oNewPerson.STD_N_FIRST_NAME = "Daniel"
oNewPerson.STD_N_FAMILY_NAME_1 = "Quinn"

.....

If oPersonChannel.CommitChanges() Is OK
Print "Changes succesfully saved"

' Eliminación de un registro de la colección (método I)
oPersonCollection.Remove(0)
If oPersonChannel.CommitChanges() Is OK
Print "Changes succesfully saved"

' Eliminación de un registro de la colección (método II)
oDeletedPerson = oPersonCollection.Item(2);
oDeletedPerson.Remove();

' Definición de una subcolección
oPersonSubCollection=oPersonCollection.Find("STD_N_FAMILY_ NAME 1
<> "Brown");
oPerson = oPersonSubCollection.Item(0);
Print oPerson.STD_N_FAMILY_NAME_1

' Cierre de la sesión
oSession.Logoff();
```

Por otra parte, también se pueden emplear interfaces no estándar, tales como el método para el cálculo de la edad media de las personas por países. En este caso, el Meta4Object de trabajo es PERSON_AGE, sobre el que se va a aplicar el método para calcular la edad media de las personas por género y país de origen al que da acceso la aplicación Person.xls (CALC_AVERAGE_AGE). Este método está definido a nivel de colección de registros.

Producto Meta4 como servidor DCOM

En ocasiones es necesario realizar llamadas a Meta4Objects desde clientes que no se encuentran alojados en la misma máquina. Por eso, disponemos de las ventajas que la tecnología DCOM proporciona para que el producto Meta4 actúe como servidor DCOM.

DCOM permite el acceso a componentes ActiveX que se encuentran alojados en una máquina distinta a la máquina desde la que se realizó la llamada.

La ventaja que ofrece es que, a partir de un cliente bastante ligero, se pueden realizar operaciones directamente sobre el producto Meta4 sin cargar en exceso los clientes, debido a que todo el proceso recae sobre el servidor de aplicaciones.

Para realizar este proceso, se debe partir de un cliente ligero, (VisualBasic, Excel...) residente en una máquina remota. Esta máquina debe tener registrada la biblioteca OLEDSI para posibilitar la comunicación con el servidor de aplicaciones. El estándar DCOM impone una limitación: al estar cada parte localizada en máquinas distintas, en cada parte tiene que haber archivos ejecutables (.EXE). Por tanto, en la parte servidora existirá un archivo ejecutable (M4DCOMSVR.EXE) que se encargará, según petición del cliente, de cargar en memoria la dll CHWRAP. Este proceso se hace de manera transparente para el usuario y siguiendo los siguientes pasos:

1. El cliente realiza una petición al ejecutable M4DCOMSVR.EXE (remoto) para que cargue en memoria una determinada biblioteca.
2. El servidor carga en memoria dicha biblioteca, devolviendo el puntero donde se almacena la biblioteca al cliente que ha realizado la llamada.
3. En este momento, el cliente ya está capacitado para establecer un dialogo con la biblioteca deseada.

Otra limitación es que sólo es posible la conexión de un único cliente en remoto. Esto se debe a que la biblioteca CHWRAP no es *multithreading*. Este problema se soluciona creando un ejecutable por cada cliente que se conecte al servidor. De esta manera, cada cliente tiene su propio Meta4Object Engine en la máquina servidora y no se producen problemas de datos incorrectos, garantizando la concurrencia de datos.

Para conseguir esto, es preciso configurar como *SingleUse* el ActiveX ejecutable que se encuentra en el padre (M4DCOMSRV.EXE). Así se garantiza que para cada instancia se cree un ejecutable distinto.

Por tanto, la situación final es que, por cada cliente conectado, existe un ejecutable en la máquina servidora, trabajando de manera independiente cada uno con un Meta4Object Engine. Una vez terminada la sesión de trabajo, se

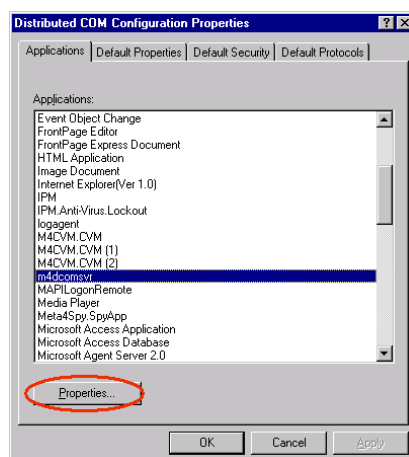
liberará tanto el Meta4Object Engine como el espacio en memoria para realizar la comunicación entre el cliente y el servidor.

Otro requerimiento de configuración antes de comenzar a trabajar con una aplicación con DCOM es configurar el acceso a esta aplicación. Estas operaciones se realizan mediante una herramienta que ofrece Windows. La herramienta es **Dcomcnfg**, que se encuentra en la carpeta `C:\WINNT\system32`.

Desde esta herramienta puede conceder permisos a los usuarios que van a tener acceso a la aplicación DCOM (m4dcomsvr) a la que se está haciendo la llamada. Para ello, siga estos pasos:

1. Una vez abierta la aplicación Dcomcnfg, aparece una primera ventana donde se muestra una lista con todas las aplicaciones. En esta lista, busque la aplicación m4dcomsvr, selecciónela y haga clic en el botón **Properties**.

Figura 12. Ventana Distributed COM Configuration Properties



2. Haga clic en la pestaña **Security**.
3. Seleccione los distintos botones de opción para conceder permisos de acceso, ejecución o configuración y haga clic en el correspondiente botón **Edit**.
4. Desde la ventana **Registry Value Permissions** puede, siempre que sea preciso, agregar nuevos usuarios. Haga clic en **Add**. Seleccione los usuarios o grupos de usuarios y asigne un tipo de acceso, variando éste en función del permiso que se asigne (acceso, ejecución o configuración). Una vez que haya terminado de añadir usuarios, haga clic en el botón **Ok**.

Una vez realizados todos estos pasos, puede realizar llamadas a objetos del producto Meta4 desde clientes remotos.

Soporte de M4API para LATE BINDING

Las M4API no sólo son accesibles desde aplicaciones que faciliten acceso a ActiveX, también lo son desde aplicaciones y lenguajes que sólo implanten *Late Binding* (enlace a posteriori). Entre estos lenguajes cabe destacar VBScript, JScript, páginas ASP, etc. Esto permite que, desde páginas HTML, se puedan ejecutar M4API que facilitan el acceso al producto Meta4.

Para crear una instancia de un Meta4Object según este modelo, hay que utilizar los métodos que ofrezca el lenguaje que se va a utilizar. Con Visual Basic, la creación de M4API_SESSION es así:

EJEMPLO

```
Dim obj as Object
Set obj=CreateObject( "M4API.0")
```

Para crear la M4API específica, se escribe 'M4API.' delante del nombre del Meta4Object en concreto:

EJEMPLO

```
Dim ref as Object
Set ref=CreateObject ( "M4API.SCO_API_HR")
```
